

JavaScript 자료구조 심화

학습 체크리스트

- ☐ 고차함수(forEach, filter, map, reduce)의 매개변수 및 콜백을 통한 배열 제어 여부
- ☐ 여러 고차함수들을 직관적으로 연쇄 구동하는 메서드 체이닝(Method Chaining) 기법 구현 여부
- ☐ 변수 재할당(참조 공유)과 원소의 물리적 공간을 분리하는 복사(Copy)의 차이 인지 여부
- ☐ 얕은 복사(Shallow Copy)와 깊은 복사(Deep Copy)의 다차원 객체 오염 한계성 비교 및 방어 코딩 적용 여부

고차함수의 개념

고차함수 (Higher-Order Function)

함수를 인자로 전달받거나 함수를 결과로 반환하는 특수한 함수로, 자바스크립트에서는 배열을 순회 가공하기 위한 내장 API들이 고차함수로 구현됨

- **값으로서의 함수:** 자바스크립트의 함수는 일급 객체이므로 다른 함수의 인자로 전달 가능함
- **동적 제어:** 데이터 처리 방식을 호출 시점에 콜백 함수로 결정하여 유연성을 제공함
- **선언형 패턴:** 데이터를 어떻게(How) 가공할지보다 무엇을(What) 할지에 집중함

4대 배열 고차함수 (forEach & map)

- **forEach()** - 단순 순회 및 조회
 - 배열의 모든 요소를 순차적으로 순회하며 전달받은 콜백을 실행함
 - 별도의 반환값(ReturnValue)이 없으며, 주로 단순 출력이나 부수 효과(Side Effect)를 일으킬 때 사용함
- **map()** - 요소 전체 변형
 - 배열의 모든 요소를 가공하여 **동일한 크기의 새로운 배열**을 생성함
 - 원본 배열은 전혀 수정하지 않는 안전한 비파괴적 메서드임

forEach & map 실전 코드

```
1  const items = [10, 20, 30];
2
3  // 1. forEach: 단순 조회 및 출력
4  items.forEach(n => console.log(n));
5
6  // 2. map: 각 원소를 2배로 가공하여 새로운 배열 반환
7  const doubled = items.map(n => n * 2); // [20, 40, 60]
```

- 비파괴적 특징: `map()` 을 실행한 후에도 원본 `items` 배열은 원본 그대로 유지됨
- 용도 구분: 단순 출력/저장 등은 `forEach`, 새 가공 배열이 필요하면 `map` 을 사용함

4대 배열 고차함수 (filter & reduce)

- filter() - 조건부 요소 선별
 - 콜백 함수의 판정 조건이 `true` 를 반환하는 요소들만 추출하여 새로운 부분 배열을 생성함
 - 조건에 만족하는 원소가 배열에 없으면 빈 배열(`[]`)을 반환함
- reduce() - 단일값 압축 수렴
 - 배열을 돌며 누적값(`acc`)과 현재값(`cur`)을 연산하여 단 하나의 최종값으로 압축함
 - 누적 연산의 시작점이 되는 두 번째 인자(초기값) 설정이 매우 중요함

filter & reduce 실전 코드

```
1  const items = [10, 20, 30];
2
3  // 1. filter: 25보다 큰 요소만 걸러내어 부분 배열 생성
4  const filtered = items.filter(n => n > 25); // [30]
5
6  // 2. reduce: 모든 요소를 합산하여 하나의 값으로 압축
7  const sum = items.reduce((acc, cur) => acc + cur, 0); // 60
```

- 부분 수집: `filter` 는 원본 배열의 크기보다 작거나 같은 크기의 배열을 반환함
- 누적 연산: `reduce` 의 `acc` (accumulator)에는 각 반복마다 계산된 중간 결과가 계속 누적됨

메서드 체이닝 (Method Chaining)

메서드 체이닝 (Method Chaining)

객체나 배열 메서드가 새로운 값을 반환할 때, 그 반환값에 연이어 다른 메서드를 꼬리 묶어 호출하는 표현 기법

- **임시 변수 제거:** 가공 과정에서 생성되는 중간 상태 변수들을 획기적으로 줄여줌
- **가시적 흐름:** 데이터가 정제되고 변형되는 파이프라인 흐름을 위에서 아래로 직관적으로 추적할 수 있음
- **코드 집약성:** 동일한 데이터 소스에 대한 여러 가공 연산을 유기적으로 결합함

메서드 체이닝 실전 코드

```
1  const scores = [45, 82, 60, 95, 30];
2
3  // 60점 이상 필터링 -> 각각 10점씩 가산 -> 최종 합산
4  const passedTotal = scores
5    .filter(score => score ≥ 60) // 1. [82, 60, 95]로 선별
6    .map(score => score + 10)    // 2. [92, 70, 105]로 가공
7    .reduce((acc, cur) => acc + cur, 0); // 3. 최종 누적합 구하기
8
9  console.log(passedTotal); // 267
```

- 파이프라인 구조: 각 단계가 이전 단계의 결과 배열을 전달받아 유기적으로 처리함
- 안전한 가공: 원본 `scores` 배열은 보존되며, 최종 결과값만 간결하게 도출됨

참조 타입과 복사의 필요성

참조 타입 복사 (Reference Copy)

객체나 배열 같은 참조 타입은 메모리 힙(Heap) 공간에 실제 데이터를 보관하고 변수에는 해당 주소값을 담으므로 복사 시 참조 범위를 제어해야 함

- **참조(Address) 전달:** 자바스크립트에서 객체나 배열은 값 자체가 아닌 주소값으로 소통함
- **의도치 않은 수정:** 하나의 참조를 여러 변수가 공유하면 예기치 못한 원본 오염이 발생함
- **불변성 지향:** 현대 프로그래밍에서는 원본을 유지하고 안전하게 복사하는 불변성이 필수임

단순 재할당 (Reassignment)

- 개념: 변수에 담긴 객체의 메모리 주소값 자체를 그대로 복제하여 대입하는 행위
- 결과: 두 변수가 물리적으로 완벽히 동일한 힙(Heap) 영역의 객체를 가리키게 됨

```
1  const userA = { name: "철수" };
2  const userB = userA; // 주소값만 그대로 공유 (재할당)
3
4  // userB의 이름을 바꾸었으나, userA의 이름도 바뀜
5  userB.name = "영희";
6
7  console.log(userA.name); // "영희" ⚠️ (동일한 주소를 공유하므로 원본 오염 발생)
8  console.log(userA === userB); // true (물리적으로 같은 객체)
```

얕은 복사 (Shallow Copy)

얕은 복사 (Shallow Copy)

객체의 1단계 깊이(Top-level) 프로퍼티만 독립된 새 메모리에 복사하고, 중첩된 다차원 객체는 기존 주소값을 그대로 복제 공유하는 방식

- **1단계 독립:** 1단계 깊이의 일반적인 단일 속성 값은 완전히 다른 주소로 격리됨
- **구현 기법:** 스프레드 연산자(`...`), `Object.assign()` 메서드 사용
- **다차원 한계:** 내부에 중첩된 객체나 배열은 주소 복제에 그쳐 여전히 같은 대상을 가리킴

얕은 복사의 한계와 오염 예시

```
1  const original = { name: "노트북", detail: { color: "Silver" } };
2
3  // 얕은 복사 수행 (Spread 연산자 활용)
4  const copy = { ...original };
5
6  copy.name = "태블릿";           // 1단계 값 수정 -> 원본에 영향 없음
7  copy.detail.color = "SpaceGray"; // ⚠ 중첩 객체 값 수정 -> 원본도 같이 변경됨!
8
9  console.log(original.name);      // "노트북" (안전함)
10 console.log(original.detail.color); // "SpaceGray" ⚠ (오염 발생!)
```

- 1단계(`name`): 정상적으로 복사되어 원본 보존 성공
- 중첩 객체(`detail`): 참조 주소만 복제되었으므로 결국 같은 `detail` 객체를 수정하게 됨

깊은 복사 (Deep Copy)

깊은 복사 (Deep Copy)

1단계뿐만 아니라 내부에 중첩된 다차원 객체까지 싹 추적하여 완전히 새로운 메모리 영역에 개별 복제하는 안전한 방식

- **완전 격리:** 모든 차원의 중첩 객체까지 개별 복제되므로 어떠한 상황에서도 원본이 오염되지 않음
- **내장 API:** 최신 브라우저와 Node.js 환경에서 표준 내장 지원하는 `structuredClone()` API 사용 권장
- **이전 방식의 제약:** `JSON.stringify` 활용법은 함수나 `undefined` 속성이 누락되는 한계가 있음

깊은 복사 실전 코드

```
1  const originalObj = { name: "노트북", detail: { color: "Silver" } };
2
3  // 안전한 표준 깊은 복사 수행 (structuredClone API 구동)
4  const deepCopy = structuredClone(originalObj);
5
6  // 중첩 객체 내부 수정
7  deepCopy.detail.color = "SpaceGray";
8
9  console.log(originalObj.detail.color); // "Silver" (오염 없이 완전히 격리 보존!)
10 console.log(deepCopy.detail.color);    // "SpaceGray" (독립된 변경 사항 적용)
```

- **안전성 확보:** 복합 다차원 데이터를 다룰 때 부수 효과를 사전에 완전 차단함
- **structuredClone():** 빠르고 표준 규격을 완벽히 따르는 자바스크립트 공식 깊은 복사 도구